

CM3070 Computer Science Final Project

Final Report

Deep Learning-based Breast Cancer
Detection System with Explainability

1. An Introduction (363 / 1000 words)

- i. The project concept: Deep learning-based breast cancer detection system with explainability
 - 1. This project provides a breast cancer detection system which is based on deep learning.
 - 2. Additionally, this breast cancer detection system offers explainability.
 - 3. Explainability refers to offering patients clear information regarding the factors that contribute to breast cancer diagnosis. If it is positive, the system explains why it is positive. If the outcome is negative, the system provides an explanation for the result.
- ii. Project template
 - 1. Course: CM3015 Machine Learning and Neural Networks
 - 2. Idea Number: Project Idea Title 2 (Deep Learning Breast Cancer Detection)
- iii. Motivation
 - 1. Doctor's view
 - a. Doctors want to create an Artificial Intelligence Breast Cancer Detection system to complement their jobs for two reasons. First is the doctor's burnout and second is the diagnosis complement.
 - b. Due to experiencing burnout ^[1], doctors need support in their work. An AI breast cancer detection system could provide valuable assistance.
 - c. The AI breast cancer detection system can provide different aspects for diagnosis with human doctors. So multiple views for disease can improve diagnosis.
 - d. The AI breast cancer detection system offers a valuable complement to human doctors by providing alternative perspectives during the diagnostic process. By integrating multiple viewpoints, the system facilitates a more thorough examination of the disease. This approach can lead to improved diagnostic accuracy, as both human expertise and AI-driven insights are combined to deliver a more comprehensive assessment of each case.
 - 2. Patient's view
 - a. And patients also want the AI Breast Cancer Detection System. There aren't enough doctors who can care for breast cancer patients. There is chronic shortage of doctors because human doctors must retire ^[2]. AI systems can complement shortage of doctors. And the co-work of the AI detection models and human doctors can enhance clinical ability. They can detect disease more accurately and

faster than before.

3. Why is explainability needed?
 - a. That is because doctors must explain their diagnosis to their patients. If the detection system just provides diagnosis results without explanation, it is not the complete detection system. The diagnosis without explanation cannot be recognized as the real diagnosis by both patients and peer doctors.

2. A literature Review (1015 / 2500 words)

- i. Deep learning-based breast cancer detection in mammography ^[3]
 1. Summary
 - a. A breast cancer detection model using a modified Efficient Net V2 with enhanced attention mechanisms was proposed. This models classification and localization capabilities demonstrated robust performance.
 2. What is good
 - a. One of the state-of-the-art paper in the deep learning breast cancer detection models
 - b. Model evaluation metrics are proposed. For example. AUROC (Area Under the Receiver Operating Characteristic Curve), Sensitivity, NPV (Negative Predictive Value), LLF (Lesion Localization Fraction)
 - c. Image Classifier based on Encoder-Decoder Model with Transformer is proposed.
 3. What to improve
 - a. No Code is proposed.
- ii. A curated mammography data set for use in computer-aided detection and diagnosis research ^[4]
 - a. Summary
 - i. Curated Breast Imaging Subset of Digital Database for Screening Mammography (CBIS-DDSM) is proposed
 - b. What is good
 - i. The problem which original DDSM dataset was solved with several image processing techniques.

- ii. The importance of standard dataset CBIS-DDSM is proposed
- c. What to improve
 - i. Actual model research for this dataset is needed
- iii. Deep Learning with Python ^[5]
 - 1. Summary
 - a. Deep learning
 - i. Deep learning is a subfield of machine learning. Deep refers to an idea of successive layers of representation.
 - b. Mathematical building blocks of neural networks.
 - i. Deep learning models are composed of the chain of tensors operations.
 - ii. All tensor operations are differentiable
 - c. Introduction to tensor flow, pytorch, jax, keras
 - i. Tensor flow, pytorch , jax and keras are popular AI frameworks.
 - d. Classification and regression
 - i. The most common machine learning tasks are classification and regression.
 - e. Fundamentals of machine learning
 - i. Generalization is the purpose of machine learning. Generalization is to perform accurately on never-before-seen inputs.
 - f. The universal workflow of machine learning
 - i. Defining the task
 - 1. You can do good work only with a deep understanding of the context of your problem.
 - 2. Framing the problem
 - a. Input: What will your input data be?
 - b. Output target: What are you trying to predict?
 - c. Type: What type of machine learning task? Binary classification? Multiclass classification? Scalar regression?
 - 3. Collecting a dataset
 - a. A model's ability to generalize comes from the data it is trained on.

- b. The information that data matters more than algorithms was made in a 2009 Google paper "The Unreasonable Effectiveness of Data".
 - c. Investing in data annotation infrastructure
 - i. The data annotation process will determine the model quality.
 - ii. Annotation by yourself, crowdsourcing platform, company are available solutions.
 - d. Beware of non representative data
 - i. The training data should be representative. Then the machine learning model can only make sense of inputs that are similar to what data they've trained by.
 - ii. The problem of sampling bias: common case of non representative data is sampling bias.
- 4. Understanding your data
 - a. You should explore and visualize data to gain insights.
 - b. If the classes are not equally represented, then you will need to account for this imbalance.
- 5. Choosing a measure of success
 - a. For balanced classification problems, where every class is equally likely, accuracy and area under curve (AUC of the receiver operating characteristic (ROC) are common metrics.
 - b. For class imbalanced problems, you can use precision and recall or a metric that counts false positives, true positives, false negatives, and true negatives.
- ii. Develop a model
 - 1. Preparing the data
 - a. Data processing means to make the raw data more amenable to neural networks.
 - b. Vectorization, normalization, and handling missing values are important steps.
 - 2. Choosing an evaluation protocol
 - a. 3 common evaluation protocols
 - i. A hold-out validation set

- ii. K-fold cross validation
 - iii. Iterated k-fold validation
- 3. Beating a baseline
 - a. To develop a small model that is capable of beating a simple baseline
- 4. Scale up: developing a model that overfits
 - a. The universal tension in machine learning is between optimization and generalization. The ideal model is one that stands right at the border between underfitting and overfitting.
- 5. Regularizing and tuning your model
 - a. Repeatedly modify your model, train it, evaluate on your validation data.
- iii. Deployment
 - 1. Explaining your work to stakeholders and setting expectations
 - 2. Shipping an inference model
 - a. Deploying a model as a REST API
 - b. Deploying a model on a device
 - c. Deploying a model in the browser
 - d. Inference model optimization
 - 3. Monitoring your model in the wild
- g. A deep dive on keras
 - i. Keras offers a different workflow. Fit function is to train, evaluate function is to evaluate.
- h. Image Classification
 - i. ConvNet is good at computer vision tasks.
- i. ConvNet Architecture Patterns
 - i. Vision Transformer is an alternative to ConvNets for large computer vision datasets.
- j. Interpret what ConvNets learn
 - i. Grad-CAM visualize what areas are related with classification
- k. Image Segmentation

- i. Image Segmentation is one of the important computer vision tasks.
 - l. Object Detection
 - i. Object Detection locates objects within bounding boxes.
 - m. Timeseries Forecasting
 - i. When the data have orders, recurrent networks are good.
 - n. Text classification
 - i. Embedding is an efficient way to transform token ID info into latent space.
 - o. Language models and transformer
 - i. Transformer is a sequence modeling architecture that uses attention as the only mechanism to pass information across a sequence.
 - p. Text generation
 - i. Transformer architecture is an important ingredient of Large Language Models.
 - q. Image Generation
 - i. Image generation is done by latent spaces that capture statistical information about a image datasets.
 - r. Best practices for real world
 - i. An ensemble technique can often enhance model prediction ability
 - s. The future of AI
 - i. The evolution of model growing system can be an artificial general intelligence
 - t. Conclusions
 - i. Transformer uses attention to transform vector into another representation
2. What is Good
- a. It is good for Comprehensive understanding of deep learning. ConvNet and Transformer are two important architectures in computer vision tasks.
 - b. It is good for Vision Transformer understanding.
 - c. Plenty of Coding examples which are very useful for project development
3. What to improve
- a. This book's focus is on general deep learning, so medical appliance of deep

learning is not much in this book

3. Design (439 / 2500 words)

a. Data Collection

i. I used CBIS-DDSM Dataset from Kaggle

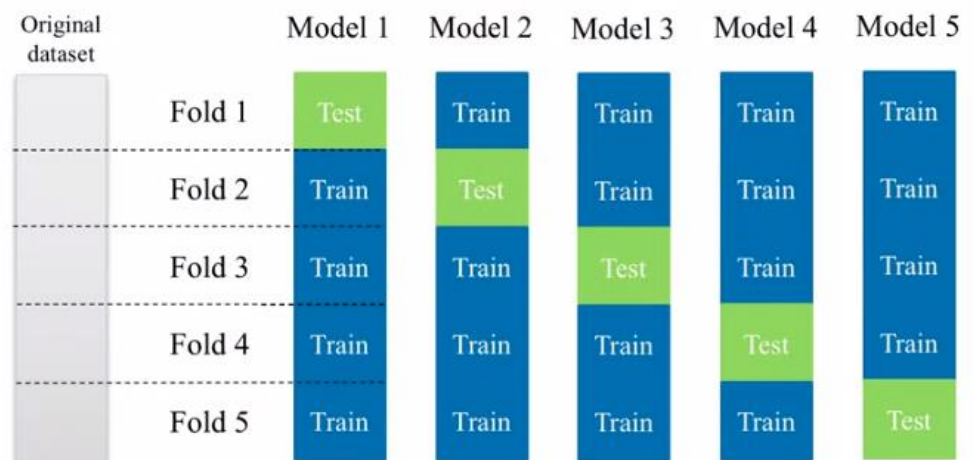
1. <https://www.kaggle.com/datasets/awsaf49/cbis-ddsm-breast-cancer-image-dataset/data>

b. Diagram of Training Stage

i. 5 Fold Cross Validation Model

1. My project design is based on K-Fold Cross Validation Model. From the book deep learning with python, K-fold cross validation is suggested for few samples for hold-out validation. That is why I selected K-Fold Cross Validation. to obtain a more robust generalization, I additionally evaluated the model using 5-fold cross validation. The dataset was split into five stratified folds. In each run, four folds were used for training and one-fold for validation, rotating until every fold is used as a validation dataset.

Cross-validation Example (5-fold)

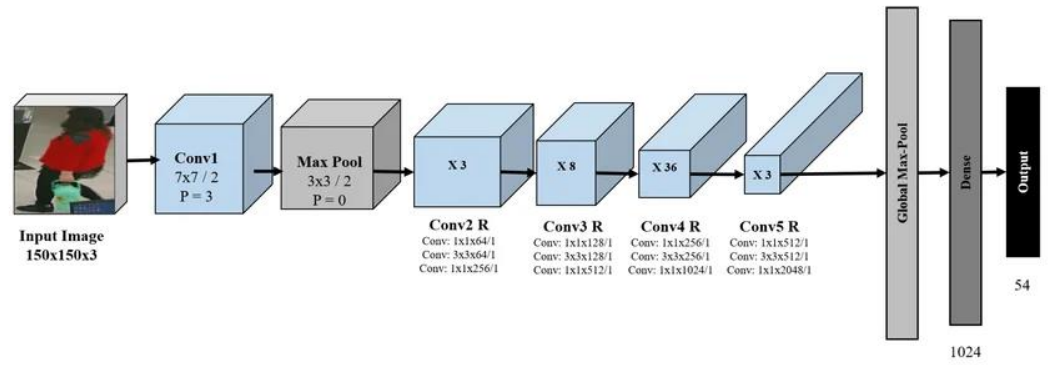


a.

< 5 Fold Cross Validation Model^[6]>

ii. Resnet 152

1. I used ResNet152 with all layers trainable. So there is no freezed layer in resnet model. I first trained my model with lower layers as freezed. But the accuracy performance is not enhanced. So I did not used transfer learning. I started with ResNet152_weights.IMAGENET1K_V2 as an initial weight. ResNet152 generally achieves higher performance than smaller ResNet model list ResNet 18.



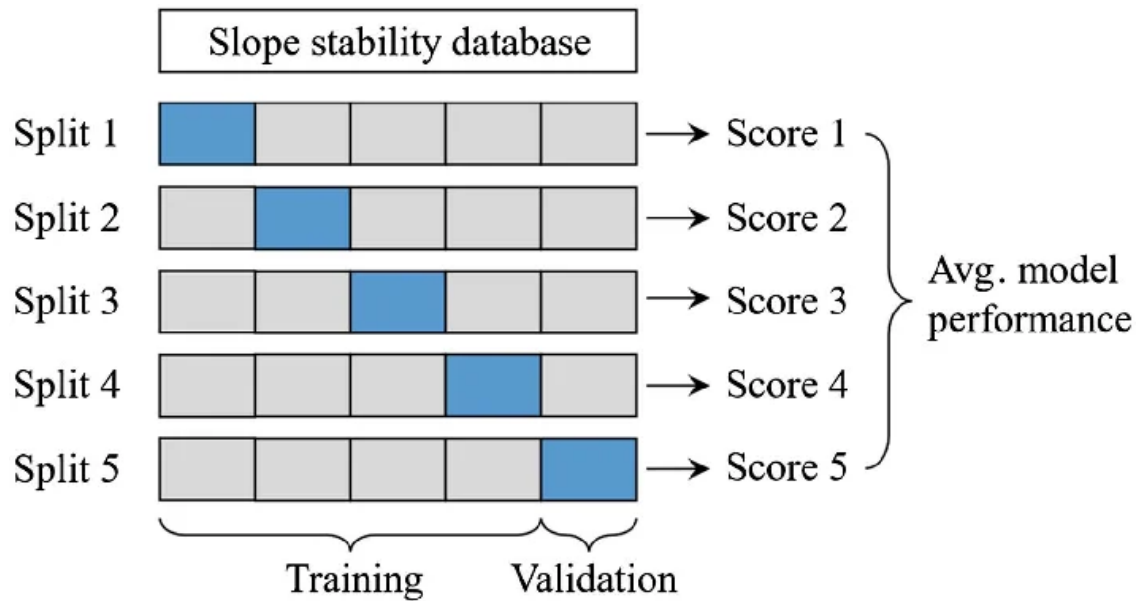
a.

< ResNet 152 Model Architecture [7]>

c. Diagram of Inference Stage

i. 5-Fold Ensemble Predictions: Average Model Performance Strategy

1. After the training stage, my new model will be used as a classifier. To utilize 5 fold structure, I employed an ensemble strategy based on 5-fold cross validation model. Instead of selecting a single trained model, the predictions from all five fold models were combined at inference time to produce a final decision



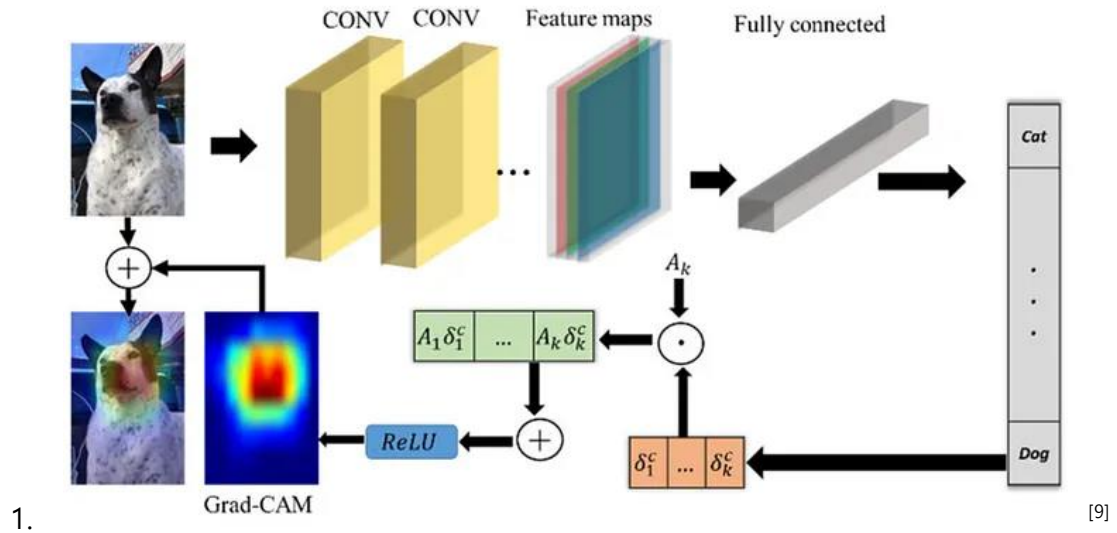
a.

< 5 Fold Ensemble Test Method [8]>

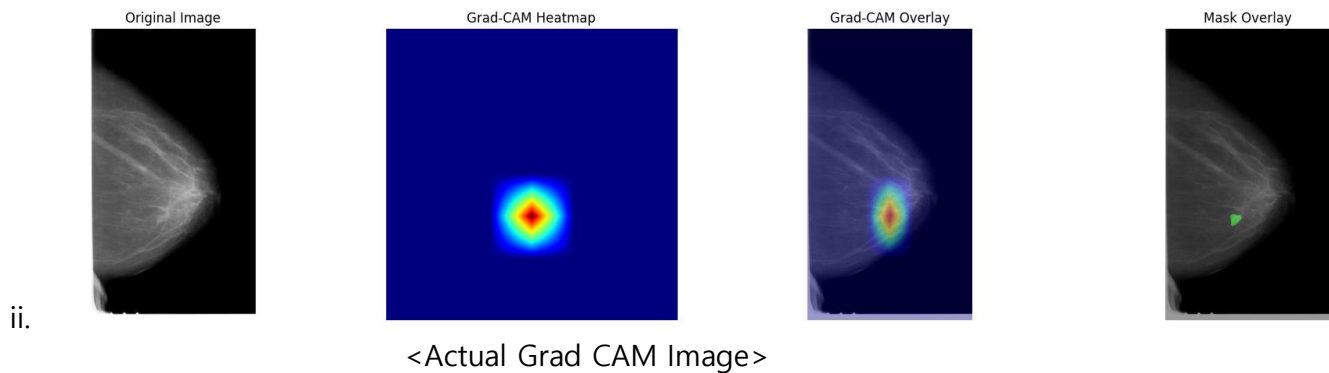
2. At test time, a given input image was passed through all five trained models, producing five output logits (probability scores) corresponding to the two classes. These outputs were first averaged to aggregate logits for one sample. For example, each test sample has two logits. One is logit for malignant, another is logit for benign. So after this stage, tensor with dimension (samples, classes) will be created. then using this tensor, make a decision for class (malignant or benign) by torch.max. For each sample, the class (Benign or Malignant) with the highest confidence score across all models were selected as the final prediction.

d. Explainability

- i. GradCAM technique is be used to gain explainability. This feature is activated from Convolutional Neural Network's feature maps.



True Positive 'Malignant' Sample (True: Malignant, Predicted by Ensemble: Malignant)



e. Workplan

Item	Due Date
Data Collection	December 19, 2025 ~ December 31, 2026
Model Architecture Development	January 1, 2026 ~ January 20, 2026
Model Training	January 21, 2026 ~ February 15, 2026
Model Evaluation	February 16, 2026 ~ March 10, 2026
Final Report	Mar 11, 20206 ~ March 23, 2026

4. Implementation (1740 / 2500 words)

- a. An architecture

- i. My project goal is to make a breast cancer image classifier. So, my model is an image classifier. The neural network of this model is a ResNet152 architecture. transfer learning is not used. So all layers are not freezed.
- ii. All code is access from git.
 1. Final_Project.ipynb
 - a. https://github.com/stanley-kim/CM3070_Final_Project/tree/Final_Project_260323
- iii. and the code is initially based on this implementation^[10].

	Prototype Architecture (5 Fold Resnet152)
Transform for train and test dataset	<pre> #define train_transform and val_test_transform # Resize: resizes the input image to a fixed size of 224x224 pixels # Grayscale(num_output_channels =3): Converts the grayscale images to a 3 channel format. 3 input channel is used for pre-trained model like ResNet # RandomHorizontalFlip(p=0.5): Flips the image horizontally with a 50% probability. # RandomVerticalFlip(p=0.5): Flips the image vertically with a 50% probability. #RandomRotation(15): Rotates the image by an anlge up to 15 degrees. # RandomAffine: Rotation up to 15 degrees, Translation up to 20% width and height, Scale from 80% to 120% # ColorJitter: Changes the brightness and Contrast of the image randomly by up to 20% # ToTensor: Transform the image to a PyTorch Tensor # Normalize: Normalize the Tensor Image with the Pre-Trained ImageNet Images Mean and Standard Deviation. train_transform = transforms.Compose([transforms.Resize((224, 224)), transforms.Grayscale(num_output_channels=3), transforms.RandomHorizontalFlip(p=0.5), transforms.RandomVerticalFlip(p=0.5), # Add RandomVerticalFlip for more augmentation transforms.RandomRotation(15), transforms.RandomAffine(degrees=15, translate=(0.2, 0.2), scale=(0.8, 1.2)), transforms.ColorJitter(brightness=0.2, contrast=0.2), transforms.ToTensor(), transforms.Normalize(mean=[0.485, 0.485, 0.485], std=[0.229, 0.229, 0.229])]) val_test_transform = transforms.Compose([transforms.Resize((224, 224)), transforms.Grayscale(num_output_channels=3), transforms.ToTensor(), transforms.Normalize(mean=[0.485, 0.485, 0.485], std=[0.229, 0.229, 0.229])]) </pre>

Class which contains image path and label	<pre> #define class MammogramDataset class MammogramDataset(Dataset): def __init__(self, df, transform=None): self.df = df.reset_index(drop=True) self.transform = transform self.label_map = {'benign': 0, 'malignant': 1} def __len__(self): return len(self.df) def __getitem__(self, idx): row = self.df.loc[idx] img = Image.open(row['image_path']).convert('L') if self.transform: img = self.transform(img) label = self.label_map[row['label']] return img, label </pre>
Model is based on ResNet152	<pre> # resnet152 with IMAGENET1K_V2 model model_fold = models.resnet152(weights=ResNet152_Weights.IMAGENET1K_V2) # Load pre-trained weights </pre>
Unfreeze all layers of ResNet152	<pre> # Unfreeze layers for fine-tuning def unfreeze_model(model, layer1_requires_grad=False, layer2_requires_grad = True, layer3_requires_grad = True, layer4_requires_grad = True,): for param in model.parameters(): param.requires_grad = False if layer1_requires_grad: for param in model.layer1.parameters(): param.requires_grad = True if layer2_requires_grad: for param in model.layer2.parameters(): param.requires_grad = True if layer3_requires_grad: for param in model.layer3.parameters(): param.requires_grad = True if layer4_requires_grad: for param in model.layer4.parameters(): param.requires_grad = True for param in model.fc.parameters(): param.requires_grad = True unfreeze_model(model_fold, layer1_requires_grad=True, layer2_requires_grad=True, layer3_requires_grad=True, layer4_requires_grad=True) </pre>
Fully Connected Layer for	<pre> # Modify the final layer for binary classification def modify_fully_connected_layer(model): </pre>

Binary Classification	<pre> num_ftrs = model.fc.in_features model.fc = nn.Linear(num_ftrs, 2) device = torch.device('cuda' if torch.cuda.is_available() else 'cpu') model = model.to(device) return model, device model_fold, device_fold = modify_fully_connected_layer(model_fold) </pre>
Optimizer is Adam	<pre> # optimizer def optimizer_for_model2(model, device, class_weights): class_weights = class_weights.to(device) # FocalLoss does not directly take class weights criterion = nn.CrossEntropyLoss(weight=class_weights) optimizer = optim.Adam([{'params': model.fc.parameters(), 'lr': 1e-4}, {'params': model.layer1.parameters(), 'lr': 1e-6}, {'params': model.layer2.parameters(), 'lr': 1e-6}, #1e-5}, {'params': model.layer3.parameters(), 'lr': 1e-6}, #1e-5}, {'params': model.layer4.parameters(), 'lr': 1e-6}, #1e-5}], weight_decay=1e-5) scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='max', factor=0.5, patience=5) return optimizer, criterion, scheduler optimizer_fold, criterion_fold, scheduler_fold = optimizer_for_model2(model_fold, device_fold, fold_class_weights) </pre>
K(5)-Fold Training	<pre> #function to train with k-fold from sklearn.model_selection import KFold def train_with_kfold(base_model, train_df, MammogramDataset, train_transform, val_test_transform, class_weights, num_epochs=40, patience=10, k_splits=5): kf = KFold(n_splits=k_splits, shuffle=True, random_state=42) fold_results = [] print(f"\n--- Starting K-Fold Cross-Validation with {k_splits} folds ---") for fold, (train_index, val_index) in enumerate(kf.split(train_df)): print(f"\n>>> Fold {fold + 1}/{k_splits}") fold_train_df = train_df.iloc[train_index] fold_val_df = train_df.iloc[val_index] # Create Datasets for the current fold </pre>

```

fold_train_dataset = MammogramDataset(fold_train_df,
transform=train_transform)
fold_val_dataset = MammogramDataset(fold_val_df,
transform=val_test_transform)

# Create sample weights for the current fold's training set
fold_class_counts = fold_train_df['label'].value_counts()
# Handle cases where a class might be missing in a fold (though
unlikely with stratify)
fold_class_weights_list = [
    1.0 / fold_class_counts.get('benign', 1.0), # Default to 1.0 to
avoid division by zero
    1.0 / fold_class_counts.get('malignant', 1.0)
]
fold_class_weights = torch.tensor(fold_class_weights_list).float()

fold_sample_weights = []
for i in range(len(fold_train_dataset)):
    img, label = fold_train_dataset[i]
    if img is not None and label is not None:
        fold_sample_weights.append(fold_class_weights[label].item())
    else:
        fold_sample_weights.append(0) # Will be ignored

fold_sampler = WeightedRandomSampler(fold_sample_weights,
len(fold_sample_weights))

# Create DataLoaders for the current fold
fold_train_loader = DataLoader(fold_train_dataset, batch_size=32,
sampler=fold_sampler, num_workers=4, collate_fn=collate_fn)
fold_val_loader = DataLoader(fold_val_dataset, batch_size=32,
shuffle=False, num_workers=4, collate_fn=collate_fn)

# initialize model, optimizer, and criterion for each fold
model_fold =
models.resnet152(weights=ResNet152_Weights.IMAGENET1K_V2) # Load pre-trained
weights
unfreeze_model(model_fold, layer1_requires_grad=True,
layer2_requires_grad=True, layer3_requires_grad=True,
layer4_requires_grad=True)
model_fold, device_fold = modify_fully_connected_layer(model_fold)
optimizer_fold, criterion_fold, scheduler_fold =
optimizer_for_model2(model_fold, device_fold, fold_class_weights)

# Train the model for the current fold
fold_train_losses, fold_val_losses, fold_train_accs, fold_val_accs =
training(
    model_fold, optimizer_fold, criterion_fold, device_fold,
fold_train_loader, fold_val_loader,

```

	<pre> scheduler_fold, num_epochs=num_epochs, patience=patience, id=f'152_fold_{fold+1}') # Load the best model saved during training for this fold model_fold.load_state_dict(torch.load(f'best_resnet_model152_fold_{fold+1}.p th')) # Evaluate on the validation set of this fold _, val_acc, _, _ = evaluate(model_fold, fold_val_loader, criterion_fold, device_fold, return_preds=True) fold_results.append(val_acc) print(f"Fold {fold + 1} Validation Accuracy: {val_acc:.4f}") avg_val_acc = sum(fold_results) / len(fold_results) print(f"\n--- K-Fold Cross-Validation Complete ---") print(f"Average Validation Accuracy across {k_splits} folds: {avg_val_acc:.4f}") return avg_val_acc, fold_results # to call the k-fold training function avg_val_acc, fold_individual_accs = train_with_kfold(model152, train_df, MammogramDataset, train_transform, val_test_transform, class_weights, num_epochs=100, patience=20, k_splits=5) </pre>
K(5)-Fold Ensemble Inference	<pre> # function to k-fold evaluation for test data kfold_models = [] num_folds = 5 # As defined in train_with_kfold device = torch.device('cuda' if torch.cuda.is_available() else 'cpu') print(f"Loading {num_folds} K-fold models...") for fold in range(num_folds): print(f"Loading model for Fold {fold + 1}...") # a. Initialize a new models.resnet152 model with pre-trained weights model_fold = models.resnet152(weights=ResNet152_Weights.IMAGENET1K_V2) # b. Call the unfreeze_model function </pre>


```

unfreeze_model(model_fold,
layer2_requires_grad=True,
layer4_requires_grad=True)

layer1_requires_grad=True,
layer3_requires_grad=True,

# c. Call the modify_fully_connected_layer function and ensure it's moved
to the determined device
# The modify_fully_connected_layer function already moves the model to
the device
model_fold, _ = modify_fully_connected_layer(model_fold)

# d. Construct the path to the saved model state dictionary for the current
fold
model_path = f'best_resnet_model152_fold_{fold+1}.pth'

# e. Load the state dictionary into the current model
model_fold.load_state_dict(torch.load(model_path, map_location=device))

# f. Set the current model to evaluation mode
model_fold.eval()

# g. Append the loaded and configured model to the kfold_models list
kfold_models.append(model_fold)

print(f"Successfully loaded {len(kfold_models)} models.")

#to make predictions with ensemble model
import numpy as np
import matplotlib.pyplot as plt

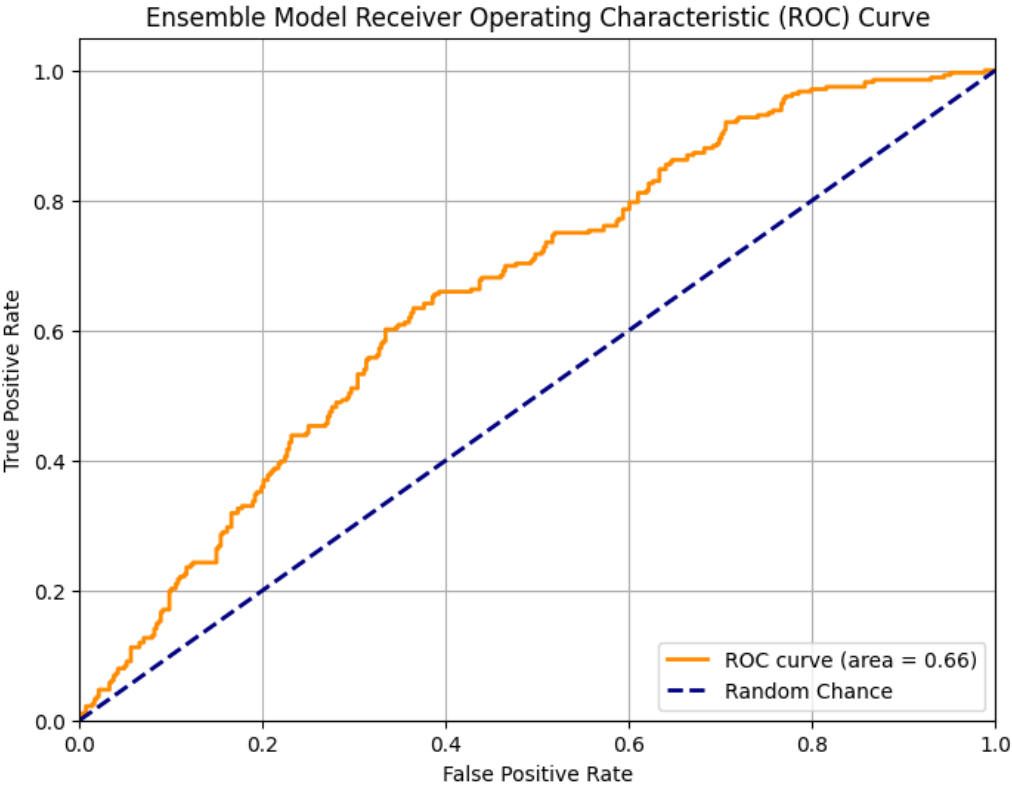
# 1. Ensemble Predictions: Calculate the average of the
all_model_predictions_tensor
# This gives the average logits from all models for each sample
ensembled_logits = torch.mean(all_model_predictions_tensor, dim=0)

# 2. Ensemble Predictions: Convert ensembled logits -> labels
_, ensemble_preds = torch.max(ensembled_logits, 1);
ensemble_preds_np = ensemble_preds.cpu().numpy()

# 3. Get true labels from the test_dataset
true_labels_np = np.array(all_labels_test)

# 4. Evaluate Ensemble Performance: Calculate the accuracy
ensemble_accuracy = np.mean(ensemble_preds_np == true_labels_np)
print(f"Ensemble Model Accuracy: {ensemble_accuracy:.4f}")

```

	Ensemble Model Accuracy: 0.6335
ROC Curve Visualization	<pre># 3. ROC Curve and AUC # Calculate probabilities for the positive class (malignant) ensemble_probs = torch.softmax(ensembled_logits, dim=1)[: , 1].cpu().numpy() # Compute ROC curve and AUC fpr, tpr, thresholds = roc_curve(true_labels_np, ensemble_probs) roc_auc = auc(fpr, tpr) # Plot ROC curve plt.figure(figsize=(8, 6)) plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'ROC curve (area = {roc_auc:.2f})') plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--', label='Random Chance') plt.xlim([0.0, 1.0]) plt.ylim([0.0, 1.05]) plt.xlabel('False Positive Rate') plt.ylabel('True Positive Rate') plt.title('Ensemble Model Receiver Operating Characteristic (ROC) Curve') plt.legend(loc='lower right') plt.grid(True) plt.show()</pre>  The figure is a Receiver Operating Characteristic (ROC) Curve plot titled "Ensemble Model Receiver Operating Characteristic (ROC) Curve". The x-axis is labeled "False Positive Rate" and ranges from 0.0 to 1.0. The y-axis is labeled "True Positive Rate" and ranges from 0.0 to 1.0. A solid orange line represents the ROC curve, which starts at (0,0) and ends at (1,1), showing a performance significantly above the diagonal. A dashed blue line represents the "Random Chance" baseline. A legend in the bottom right corner indicates "ROC curve (area = 0.66)" and "Random Chance".
GradCAM Class using layer4.conv3	<pre># GradCAM Class import torch import torch.nn.functional as F import numpy as np</pre>

```

class GradCAM:
    def __init__(self, model, target_layer):
        self.model = model
        self.target_layer = target_layer
        self.feature_maps = None
        self.gradients = None
        self.handlers = []

        self._register_hooks()

    def _register_hooks(self):
        # Register hook for feature maps (forward pass)

self.handlers.append(self.target_layer.register_forward_hook(self._save_feature_maps_hook))
        # Register hook for gradients (backward pass)

self.handlers.append(self.target_layer.register_backward_hook(self._save_gradients_hook))

    def _save_feature_maps_hook(self, module, input, output):
        self.feature_maps = output.detach()

    def _save_gradients_hook(self, module, grad_input, grad_output):
        # grad_output is a tuple, we are interested in the gradient of the
output
        self.gradients = grad_output[0].detach()

    def __call__(self, input_image, target_class=None):
        self.model.eval()
        self.model.zero_grad()

        # Forward pass
        output = self.model(input_image) # output are logits

        if target_class is None:
            # Get the predicted class if target_class is not specified
            target_class = output.argmax(dim=1).item()

        # Backward pass for the target class
        # Create a one-hot vector for the target class
        one_hot = torch.zeros_like(output, dtype=torch.float)
        one_hot[0][target_class] = 1 # Assuming batch size of 1 for simplicity

        # Compute gradients of the target class score with respect to outputs
        # Retain graph is set to True to allow subsequent backward calls if

```

	<pre> needed (e.g., for multiple Grad-CAMs) output.backward(gradient=one_hot, retain_graph=True) # Get feature maps and gradients feature_maps = self.feature_maps gradients = self.gradients # Compute alpha weights (global average pooling of gradients) # These weights represent the importance of each feature map channel weights = F.adaptive_avg_pool2d(gradients, 1) # Compute Grad-CAM heatmap # Perform a weighted sum of the feature maps using the calculated alpha weights cam = (weights * feature_maps).sum(dim=1, keepdim=True) cam = F.relu(cam) # Apply ReLU to keep only positive contributions (features that positively influence the prediction) # Normalize and resize the heatmap cam = F.interpolate(cam, size=input_image.shape[2:], mode='bilinear', align_corners=False) cam = cam - cam.min() cam = cam / (cam.max() + 1e-8) # Add small epsilon to avoid division by zero return cam.squeeze().cpu().numpy() def remove_hooks(self): for handler in self.handlers: handler.remove() # Instantiate GradCAM using the first K-fold model and its last convolutional layer # For ResNet152, the last convolutional layer in the final block (layer4) is 'conv3'. target_model = kfold_models[0] target_layer_for_cam = target_model.layer4[-1].conv3 grad_cam = GradCAM(target_model, target_layer_for_cam) </pre>
Select one true positive sample to test explainability	<pre> # find one true positive sample for Explainability Feature of the model true_positive_idx = None for i in range(len(true_labels_np)): if true_labels_np[i] == 1 and ensemble_preds_np[i] == 1: true_positive_idx = i break </pre>

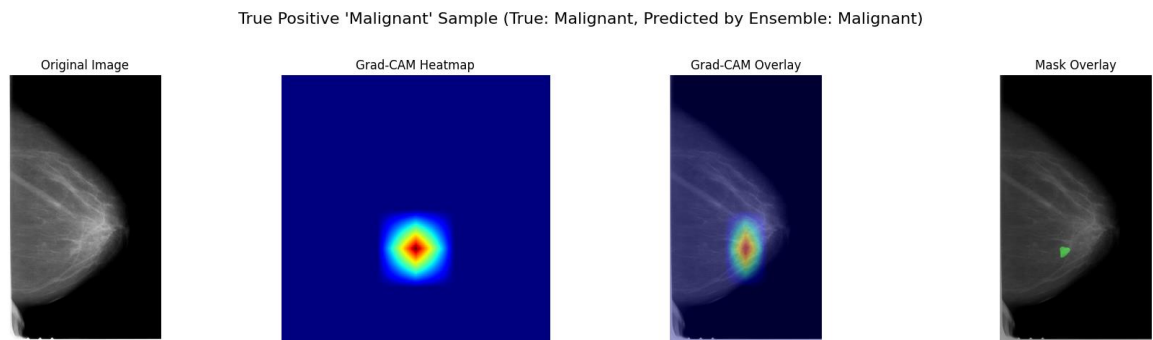
	<pre> if true_positive_idx is not None: print(f"Found first true positive 'malignant' sample at index: {true_positive_idx}") else: print("No true positive 'malignant' sample found.") </pre>
Compare Grad-CAM overlay with MAs overlay to test explainability	<pre> # compare original image, Grad-CAM Heatmap, Grad-CAM overlay, Mask overlay for one True Positive (Malignant) image import matplotlib.pyplot as plt import numpy as np from PIL import Image # 1. Retrieve the file path to the ROI mask image mask_image_path = original_test_row['mask_path'] # 2. Open the ROI mask image and convert it to grayscale mask_image = Image.open(mask_image_path).convert('L') # Get true and predicted labels as strings for the title true_label_str = original_test_row['label'] predicted_label_str = 'malignant' if ensemble_preds_np[true_positive_idx] == 1 else 'benign' # 3. Create a figure with 4 subplots plt.figure(figsize=(19, 5)) # Adjusted width for 4 subplots # 4a. Original Image plt.subplot(1, 4, 1) plt.imshow(original_image, cmap='gray') plt.title('Original Image') plt.axis('off') # 4b. Grad-CAM Heatmap plt.subplot(1, 4, 2) plt.imshow(heatmap, cmap='jet') plt.title('Grad-CAM Heatmap') plt.axis('off') # 4c. Grad-CAM Overlay superimposed_image = visualize_cam(original_image, heatmap) plt.subplot(1, 4, 3) plt.imshow(superimposed_image) plt.title('Grad-CAM Overlay') plt.axis('off') # 4d. Mask Overlay Image </pre>

```
plt.subplot(1, 4, 4)
plt.imshow(mask_overlay_image)
plt.title('Mask Overlay')
plt.axis('off')

# 5. Add a main title to the entire figure
plt.suptitle(f"True Positive 'Malignant' Sample (True: Malignant, Predicted by Ensemble: Malignant)", fontsize=16)

# 6. Adjust the layout
plt.tight_layout(rect=[0, 0.03, 1, 0.95]) # Adjust rect to make space for supitle

# 7. Display the plot
plt.show()
```



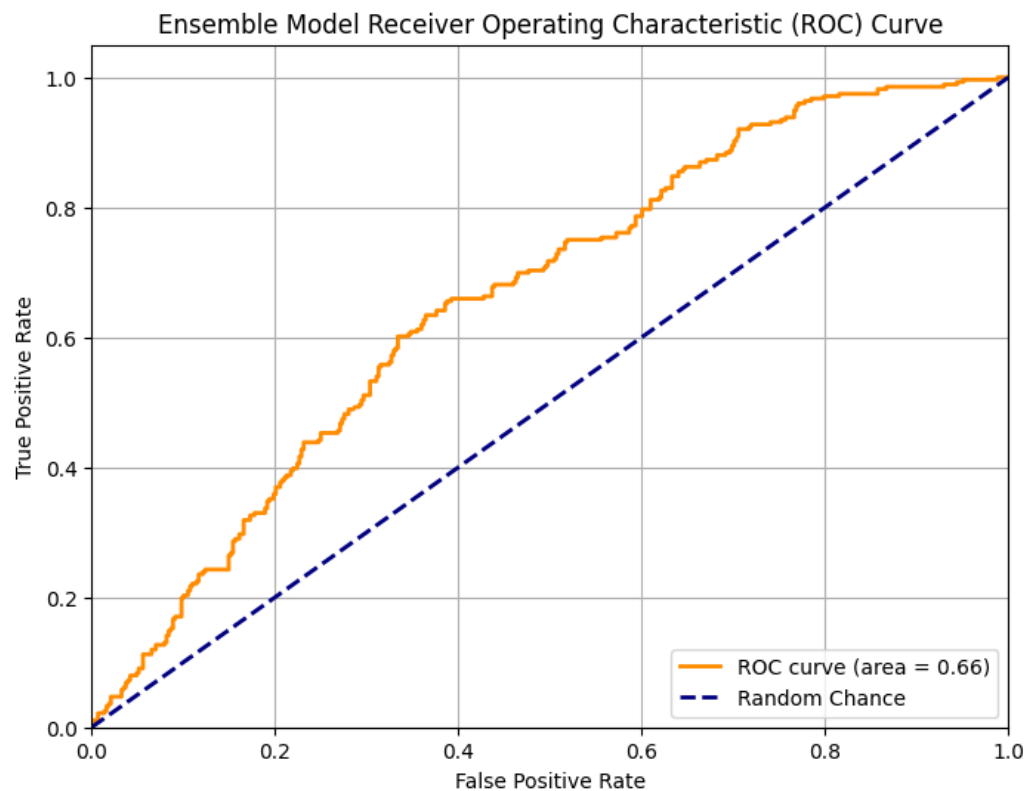
5. Evaluation (228 / 2500 words)

- i. The accuracy
 1. the accuracy is evaluated using the CBIS DDSM Test Dataset
 2. All models are trained with maximum 100 num_of_epoces and 20 num_of_patience
 3. My 5 Fold Ensemble Model is compared with 5 Fold's Average, Fold 1, Fold 2, Fold 3, Fold 4 and Fold 5

	My Model (5-Fold Ensemble Model)	5 Fold's Average	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5
Accuracy	0.6335	0.6182	0.6207	0.5895	0.6207	0.6222	0.6378

- a. 5-Fold Ensemble Model's Accuracy is more than 5 Fold's accuracy average. So the ensembling approach can be used to improve the model's overall accuracy.
- b. But 5-Fold Ensemble Model's Accuracy is similar to one fold model(Fold 5). So the ensembling effect is limited.

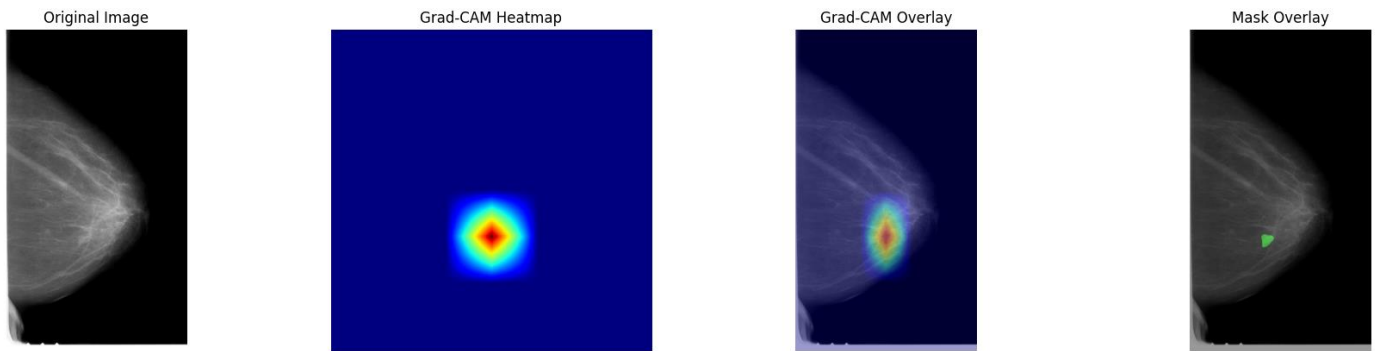
ii. ROC and AUC (Area Under the Curve)



- 1.
2. My model's AUC value is 0.66. so, it is bigger than random guess model's 0.5. so my ensemble model is a moderately effective classifier for breast cancer detection. But the AUC value is much less than a perfect classifier's 1.0, so there is some room for enhancement.

iii. Explainability

True Positive 'Malignant' Sample (True: Malignant, Predicted by Ensemble: Malignant)



1. Explainability is not calculated as a numerical value.
2. Instead, explainability can be explained by visualization.
3. The above original image is one example of a true positive (Malignant) image. So, the Grad-CAM overlay area is overlapped with a mask overlay area. So, Grad-CAM can be used for explainability features.

6. Conclusion (406 / 1000 words)

- i. This project provides a deep learning-based breast cancer detection system with explainability.
- ii. This model used K(5)-fold Cross Validation Model with ResNet15 to overcome a small dataset.
- iii. This model used 5 fold cross validation techniques for training and 5 fold ensemble predictions for inference.
- iv. The model's test data accuracy is 0.6335 which is more than the average value (0.6182) of each fold model's accuracy
- v. The explainability feature is provided by Grad-CAM technique.
- vi. To improve performance, more epochs and advanced techniques can be applied for future work.
- vii. This project presents a deep learning-based system for breast cancer detection with an emphasis on model interpretability and robustness under limited data conditions. Breast cancer remains one of the leading causes of mortality among women worldwide, making accurate and explainable computer-aided diagnostic systems critically important. The primary objective of this project is to develop a classification model that can reliably detect breast cancer from medical imaging data, which is DDSM while also providing visual explanations to support clinical decision making.
- viii. To address the challenge of a relatively small dataset, this project uses a K(5)-fold cross validation strategy combined with a Resnet152 deep convolutional neural network

architecture. Cross-Validation is a widely used technique that improves model generalization by training and validating the model on multiple subsets of the data. In this approach, the dataset is divided into five folds, with four folds used for training and one-fold for validation in each iteration. This process is repeated five times, ensuring that every data sample is used for both training and validation phases.

- ix. The experimental results demonstrate that the proposed 5 fold ensemble model achieves a test accuracy of 0.6335, which is higher than the average accuracy of 0.6182 of five fold models. This improvement indicates that ensemble prediction contributes positively to performance and validates the effectiveness of the cross-validation technique in handling limited datasets.
- x. In addition to predictive features, explainability is another focus of this project. To enhance explainability, the system integrates the Grad-CAM technique. Grad-CAM generates heatmaps that highlight the most strongly influence the model's predictions. This explainability feature allows doctors to visually interpret the model's decision process and verify whether clinically relevant regions are being considered.
- xi. There are some areas for future improvement. Model performance could be enhanced by increasing the number of training epochs, applying advanced data augmentation techniques, or exploring more sophisticated neural network architectures and optimization techniques.

7. Reference

- i. <https://www.rcr.ac.uk/news-policy/latest-updates/over-a-quarter-of-breast-cancer-doctors-set-to-retire-within-5-years-putting-patient-care-at-risk/>
- ii. <https://www.rcr.ac.uk/news-policy/latest-updates/over-a-quarter-of-breast-cancer-doctors-set-to-retire-within-5-years-putting-patient-care-at-risk/>
- iii. Wang L. "Mammography with deep learning for breast cancer detection", Front Oncol. 2024 Feb, <https://doi.org/10.3389/fonc.2024.1281922>
- iv. Lee, R., Gimenez, F., Hoogi, A., Miyake, K. K., Gorovoy, M. & Rubin, D. L. "A curated mammography data set for use in computer-aided detection and diagnosis research.", Sci Data 4, 170177 (2017). <https://doi.org/10.1038/sdata.2017.177>
- v. Françoise Collet, "Deep Learning with Python", 2018, Manning, Shelter Island
- vi. https://machine-learning-and-data-science-with-python.readthedocs.io/en/latest/_images/crossval.png
- vii. <https://www.researchgate.net/publication/357723601/figure/fig5/AS:11431281351482128@1743773568625/Overview-of-the-ResNet-152-v2-architecture.tif>

- viii. <https://www.researchgate.net/profile/Te-Pei/publication/373013772/figure/fig4/AS:11431281180476043@1691608307131/Five-fold-cross-validation-procedure-for-developing-ML-models.png>
- ix. <https://pub.towardsai.net/easy-explain-explainable-ai-with-gradcam-e187f85a697c>
- x. <https://www.kaggle.com/code/jebalimaram/cbis-ddsm-breast-tumor-cancer-detection>